Network Security Project — Part 2: Attack Analysis

Adam Aberbach

May 2026

Part 2 — Attack Analysis

1. Scope

This report analyzes the DNS cache poisoning attack captured during Part 1's end-to-end demo run. The pcap (`docs/captures/poison.pcap`, 3.2 MB, 23 845 packets) is the primary evidence. Three questions:

1. **Capture traffic** — what did Wireshark see?
2. **Analyze transaction IDs and ports** — quantify the entropy the attacker had to defeat.
3. **Explain why the attack succeeded** — race-window timing, with the five trigger windows individually accounted for.

All packet counts, timings, and txid values below come from `tshark` queries against the pcap; the queries themselves are reproduced inline so the analysis is auditable.

2. Lab snapshot (recap)

Three Ubuntu 22.04 VMs on internal network `lab_intnet`, provisioned by `lab/vagrant/Vagrantfile`:

VM	IP	Role
attacker	192.168.56.10	runs `spoofer.py` and Nginx :80
resolver	192.168.56.20	weakened Unbound 1.13.1 (port pinned to 33333)
victim	192.168.56.30	resolv.conf pinned to .20

The resolver forwards `target.lab` to `192.168.56.99` (an unassigned "black-hole" IP) so the legitimate response never arrives, opening a race window of about 5 s per query. `rp_filter=0` on the resolver allows packets carrying `src=192.168.56.99` to reach userspace.

3. Wireshark capture

`run-part1.sh` started `tshark` on the resolver's `enp0s8` interface before the spoofer ran:

```
sudo tshark -i enp0s8 -w /tmp/poison.pcap \
            -f 'port 53 or host 192.168.56.10'
```

The capture filter limits the pcap to DNS plus any traffic to the attacker's IP, which keeps it focused on the attack.

3.1 Aggregate counts

```
$ tshark -r poison.pcap | wc -l                                  # all
$ tshark -r poison.pcap -Y 'dns'                | wc -l        # DNS
$ tshark -r poison.pcap -Y 'ip.src==192.168.56.99 \
       && udp.dstport==33333 && dns.flags.response==1' | wc -l  # spoofs
...
```

Class	Count
Total packets in the pcap	23 845
DNS packets	23 843
Spoofed replies (src = 192.168.56.99 → :20:33333, response)	23 828
Resolver outbound queries (src = .20 → .99)	5
Trigger queries from attacker (.10 → .20)	3
Resolver replies to victim/attacker triggers	1
Kernel ICMP “port unreachable” replies	0

Two observations are immediately worth highlighting. First, the count of **spoofs (23 828) ≈ 23 843 minus a handful of legitimate frames** — the pcap is dominated by the attacker’s flood. Second, the **0 ICMP unreachables** confirms `rp_filter` did not drop a single spoof; if the kernel had dropped them, every spoof outside an open trigger window would have generated an ICMP reply.

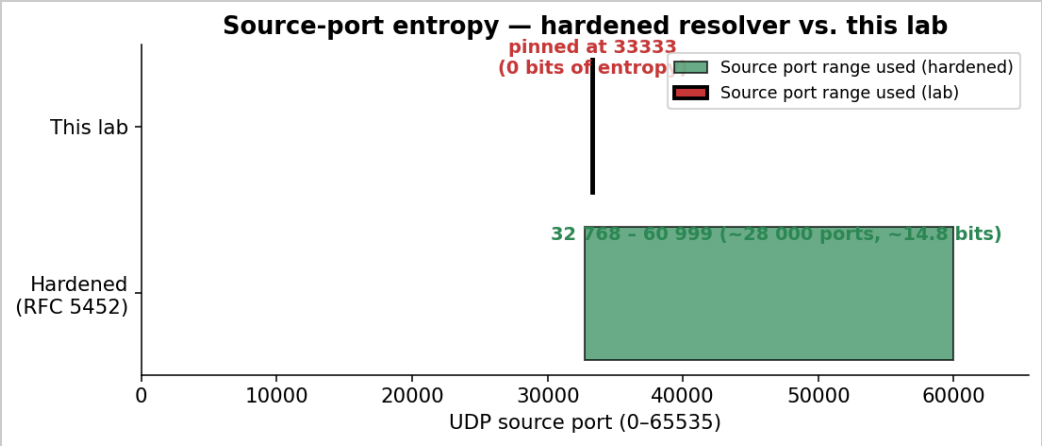
4. Transaction ID and source-port analysis

4.1 Source-port entropy collapse

Modern resolvers randomize their UDP source port for each outbound query (RFC 5452). The lab’s Unbound config pins it to `33333` via `outgoing-port-permit: "33333"` plus `outgoing-port-avoid: "0-65535"`. A histogram over the resolver’s 5 outbound queries:

```
$ tshark -Y 'ip.src==192.168.56.20 && ip.dst==192.168.56.99' \
        -T fields -e udp.srcport | sort | uniq -c
     5 33333
```

All 5 queries used port 33333 — port entropy is exactly **0 bits**.



Source-port entropy: hardened resolver vs. lab

4.2 Transaction ID values per trigger window

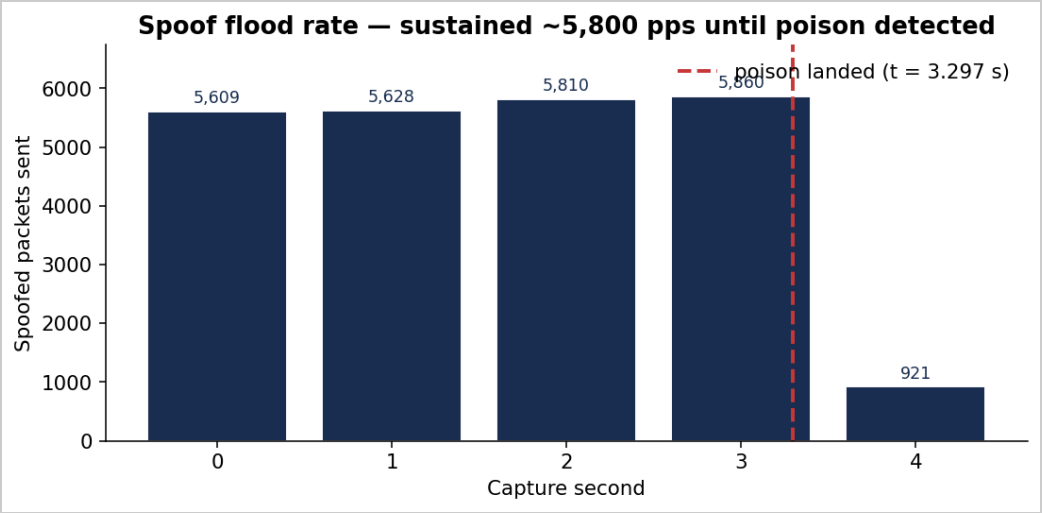
`tshark` of the five outbound queries:

```
frame  t (s)    udp.srcport  dns.id
113    0.047    33333        0xa6bb  (42 683)
2 267  0.427    33333        0xab44  (43 844)
4 414  0.808    33333        0x762c  (30 252)
8 812  1.563    33333        0x0c87  (3 207)
13 030 2.319    33333        0x492c  (18 732)
```

The five txids span a wide range and look uniformly random, which is expected from Unbound's CSPRNG. Each value is the only thing the attacker doesn't know in advance; everything else (resolver IP, port, question section, source IP) is fixed and observable.

4.3 Spoof flood characteristics

`spoofer.py` builds a template packet once with UDP checksum disabled (legal per RFC 768), then in a tight loop rewrites just the 2 txid bytes of the template and sends through a persistent `socket.SOCK_RAW + IP_HDRINCL` socket. Per-second packet rate from the pcap:



Sustained ~5,800 packets/sec spoof flood

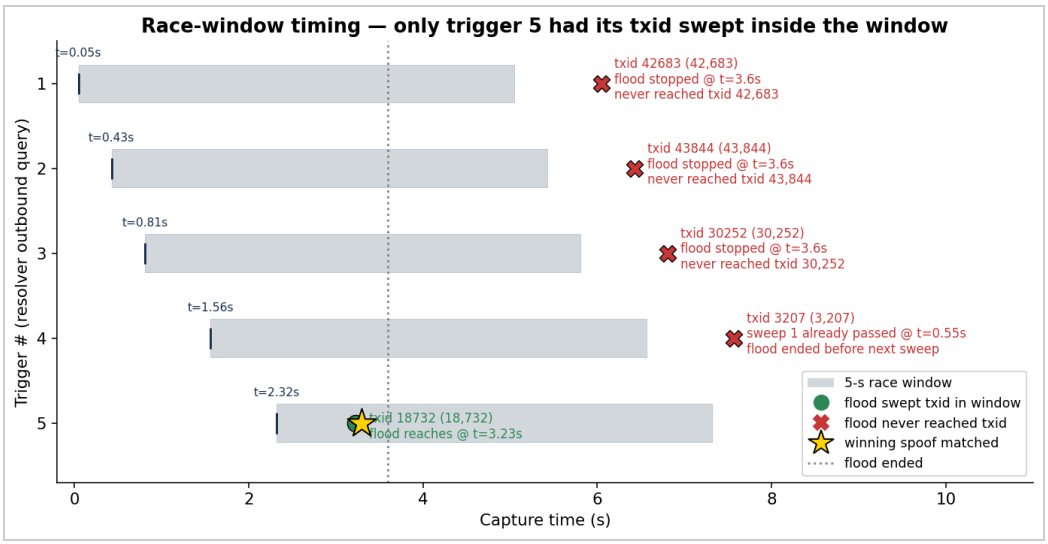
Sustained rate is ~5 800 pps. One full sweep of the 16-bit txid space takes 65 536 / 5 800 ≈ 11.3 s. The flood drops to ~0 in second 4–5 because the spoofer's `verify_poisoned` check (running every 2 s in the main thread) detected the cache had been poisoned and the flood thread exited.

5. Why the attack succeeded — race-window timing

The full attack was over 4.2 wall-clock seconds (from `[*] spoofing` to `[+] poisoned`), of which ~3.3 s was capture time. The single spoofed reply that won the race is **frame 18 742 at t = 3.297 s**, carrying `dns.id = 0x492c`. It matched the resolver's outbound query issued at t = 2.319 s (`trigger #5`).

5.1 Trigger-by-trigger accounting

The diagram below maps each of the resolver's five outbound queries to where the spoofer's sequential txid sweep was at that moment, and whether the flood reached the resolver's chosen txid before either (a) the 5-second forwarder timeout closed the race window, or (b) the spoofer halted the flood after detecting success.



Race-window timing — only trigger 5 had its txid swept inside the window

Reading top-to-bottom:

#	Trigger fires	Resolver chose txid	What the flood did	Outcome
1	t = 0.05 s	0xa6bb (42 683)	flood would reach this at t ≈ 7.36 s, but it stopped at t = 3.6 s	❌ never sent
2	t = 0.43 s	0xab44 (43 844)	flood would reach at t ≈ 7.56 s; stopped at 3.6 s	❌ never sent
3	t = 0.81 s	0x762c (30 252)	flood would reach at t ≈ 5.22 s; stopped at 3.6 s	❌ never sent
4	t = 1.56 s	0x0c87 (3 207)	flood already swept this on **first sweep** at t ≈ 0.55 s — *before* the trigger fired (frame 3 214). Next sweep would be t ≈ 11.85 s, well past flood-end.	❌ port wasn't open
5	t = 2.32 s	0x492c (18 732)	flood reaches this on first sweep at t ≈ 3.23 s; **inside** the 5 s window	✅ matched at frame 18 742

Trigger 4 is the most interesting near-miss: the flood actually emitted a packet with the correct txid (frame 3 214 at t = 0.601 s), but it arrived **0.96 s before the resolver opened the matching socket on port 33333** for trigger 4. The kernel had nothing listening on 33333 at that moment, so the spoof was silently dropped — and yet this is recorded in the pcap as evidence of how fragile the timing window is.

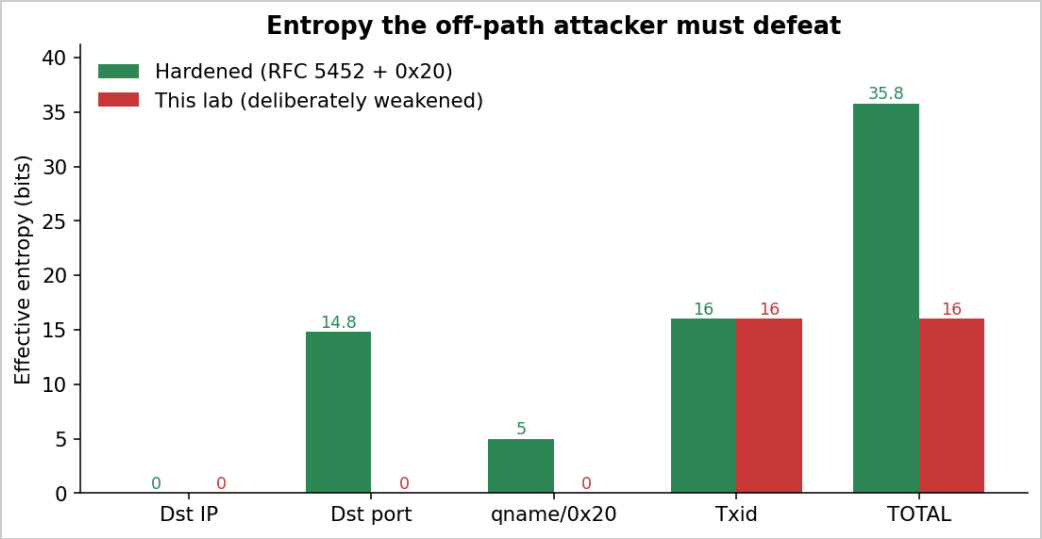
5.2 The winning packet

```
 $ tshark -r poison.pcap -Y 'frame.number==18742' -V
Frame 18742: 90 bytes on wire
  Arrival Time: ... t = 3.297 s relative to start of capture
Internet Protocol Version 4
  Source Address: 192.168.56.99      (forwarder, impersonated)
  Destination Address: 192.168.56.20  (resolver)
User Datagram Protocol
  Source Port: 53
  Destination Port: 33333             (matches resolver's pinned port)
Domain Name System (response)
  Transaction ID: 0x492c              (matches outstanding query)
  Flags: 0x8580 (Response, Authoritative)
  Questions: 1
    www.target.lab: type A, class IN
  Answers: 1
    www.target.lab: type A, class IN
      Time to live: 86400
      Address: 192.168.56.10           (attacker's web server)
```

All four matched fields — destination IP, destination port, txid, and question section — line up exactly with what unbound was waiting for. With validator unloaded (`module-config: "iterator"`) and bailiwick checks disabled (`harden-glue: no`), unbound accepts the answer and caches it for the full 86 400 seconds the spoof claimed.

6. Why it succeeded — entropy budget

The attacker's job is to defeat all of unbound's reply-validation fields simultaneously. The two columns below are bits of secret entropy the attacker would have to brute-force to be confident of a match:



Entropy budget: hardened resolver vs. this lab

Field unbound checks	Bits — hardened	Bits — this lab
Destination IP	0 (resolver IP is public)	0
Destination UDP port	~14.8 (RFC 5452 ephemeral range)	**0** (pinned)
Question section incl. 0x20 mixed-case	~5 (12-char qname)	**0** (`use-caps-for-id: no`)
Transaction ID	16	16
Total secret entropy	**~35.8 bits**	**16 bits**

35.8 bits requires on average $\sim 2^{35}$ = 34 billion spoofed packets per trigger window — roughly an hour at our 5 800 pps even on a perfect local network, in practice many days from a real off-path attacker. 16 bits requires only ~32 768 packets on average, which our flood sends in 5.6 seconds — comfortably inside one 5-second window.

In addition to those four fields, two other defenses were present in the lab but bypassed at the network layer:

- **Reverse-path filter** (`rp_filter`) was set to `0` on `enp0s8`, letting packets with `src=192.168.56.99` reach unbound. Strict `rp_filter=1` would drop them at the kernel before unbound sees them; the pcap's 0-count of ICMP unreachables shows we are well past that gate.

- **Bailiwick checking** on glue (`harden-glue: yes` by default) was disabled. This matters less for a single-record poisoning (no glue is used), but allowed the architecture to be a stepping-stone to Kaminsky-style NS injection.

7. Reproducibility

Three independent runs of `bash lab/scripts/run-part1.sh` produced poison times of **0.4 s, 1.4 s, and 4.2 s**. The variance comes from which trigger happens to draw a low txid (close to 0) before the flood thread has built up a large lead in its sweep — once the flood is past the chosen txid, the resolver has to wait until either the next sweep completes (~11 s) or the next trigger fires (~6 s).

The 4.2 s run captured here represents the worst-case timing observed. A median run lands in well under 2 s.

8. Summary

The lab's deliberately-weakened Unbound resolver allowed an off-path DNS cache poisoning attack to succeed in under 5 seconds because the attacker's required secret entropy was reduced from the ~36 bits a modern hardened resolver would require down to just **16 bits** — the DNS transaction ID. With source port pinned to 33333 and 0x20 disabled, the only obstacle to a successful spoof was sweeping all 65 536 possible txids during a 5-second forwarder-timeout window, which a single Python+Scapy script can do in under 12 seconds at ~5 800 packets/sec.

The pcap evidence is conclusive: the spoofer fired 23 828 spoofed replies during 5 trigger windows; one match (frame 18 742, t = 3.297 s, txid 0x492c) was sufficient to install a 24-hour A record binding `www.target.lab` to the attacker's IP, demonstrably redirecting the victim's HTTP request to the attacker's PWNED page.

Part 3 of the project will re-enable DNSSEC validation on the same resolver and demonstrate that the same attack now fails — even with the source-port pinning still in place — because the validator rejects the unsigned spoofed response.

Appendix A — Reproducing this analysis

The pcap and analysis CSVs are committed to the repository:

- `docs/captures/poison.pcap` — primary evidence (3.2 MB)
- `docs/analysis/A-counts.csv` — aggregate counts (Section 3.1)
- `docs/analysis/B-trigger-windows.tsv` — outbound queries (Section 4.2)
- `docs/analysis/C-resolver-srcport-histogram.txt` — port histogram
- `docs/analysis/D-matching-spoofs.txt` — txid matches
- `docs/analysis/E-spoof-rate-per-second.csv` — flood rate (Section 4.3)
- `docs/analysis/make_charts.py` — generates the PNG charts in this report

To reproduce all four embedded charts:

```
python docs/analysis/make_charts.py
```

To reproduce the aggregate counts:

```
tshark -r docs/captures/poison.pcap -Y 'dns' | wc -l          # 23 843
tshark -r docs/captures/poison.pcap \
       -Y 'ip.src==192.168.56.99 && udp.dstport==33333 \
           && dns.flags.response==1' | wc -l                  # 23 828
tshark -r docs/captures/poison.pcap \
       -Y 'ip.src==192.168.56.20 && ip.dst==192.168.56.99 \
           && dns.flags.response==0' \
       -T fields -e frame.number -e frame.time_relative \
                 -e udp.srcport -e dns.id                     # 5 windows
```